

Operadores, Expressões e Variáveis

Material de apoio para estudo e revisão

Este handout consolida o conteúdo do Módulo 2: o conceito de variável e as regras de nomeação, constantes e tipos numéricos, a avaliação de expressões, os sete operadores aritméticos, a família de atribuições compostas e a ordem de precedência. Ao final você encontra os erros mais comuns de iniciante, três desafios com gabarito e um checklist de revisão para a prova.

Ciência da Computação · Sistemas de Informação — 2º Semestre

Professores: Luciano · Rafael · Turatti

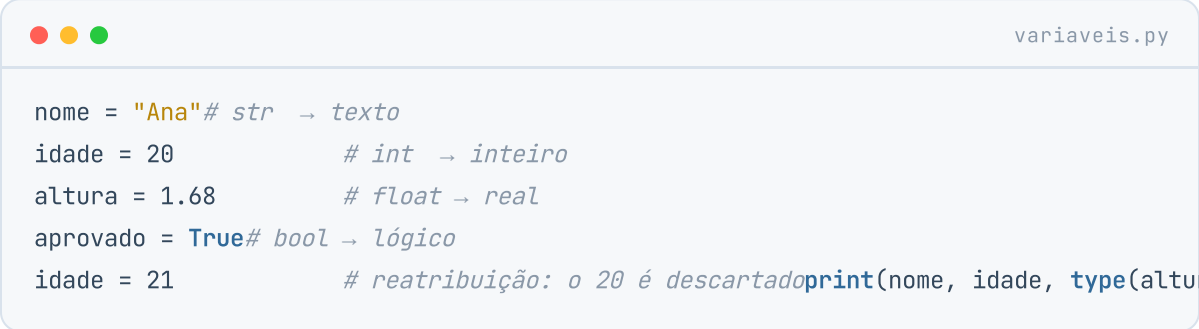


modulo2.py

```
nota = 8.5
media = (nota + 7.0) / 2
print(media)    # 7.75
```

Variáveis: nomes que guardam valores

Uma **variável** é um nome que referencia um valor guardado na memória do computador. Pense em uma caixa etiquetada: a etiqueta é o nome, o conteúdo é o valor. Em Python, a variável passa a existir no momento da **atribuição** — não há declaração prévia de tipo. A leitura correta do sinal `=` é "recebe": em `x = 5`, o valor 5 é calculado primeiro e então guardado no nome `x`. Como o próprio nome sugere, uma variável pode ser reatribuída quantas vezes for necessário; o valor anterior simplesmente deixa de ser referenciado.



```
nome = "Ana" # str → texto
idade = 20      # int → inteiro
altura = 1.68   # float → real
aprovado = True # bool → lógico
idade = 21      # reatribuição: o 20 é descartado
print(nome, idade, type(altura))
```

O tipo pertence ao **valor**, não ao nome — por isso dizemos que Python é **dinamicamente tipado**. A função `type()` revela o tipo de qualquer valor ou variável: `type(20)` devolve `<class 'int'>`. Essa liberdade exige cuidado: `idade = "vinte"` é aceito sem erro, e o nome passa a referenciar um texto. Prefira sempre nomes descritivos (`nota1` em vez de `n1`) — código é lido muito mais vezes do que é escrito.

Regra para nomes	Exemplos
Válidos: letras, dígitos e <code>_</code> , sem começar por dígito	<code>salario_bruto · nota1 · _total · mediaFinal</code>
Inválidos: dígito inicial, hífen ou espaço	<code>2nota · nome-aluno · salario bruto</code>
Proibidos: 33 palavras reservadas da linguagem	<code>class · if · for · import · while</code>
Convenção PEP 8: snake_case minúsculo	<code>media_das_notas · taxa_ipi · limite_alunos</code>
Atenção: nomes diferenciam maiúsculas	<code>Nota ≠ nota ≠ NOTA</code> (três variáveis)

02 · CONSTANTES E NÚMEROS

Constantes e tipos numéricos

Python não possui um mecanismo que **trave** um valor. Uma constante é uma **convenção**: escrever o nome todo em MAIÚSCULAS sinaliza "não altere este valor" para quem lê o código. É um contrato social da equipe, não uma regra da linguagem — tecnicamente `PI = 4` rodaria sem erro. Ainda assim, a convenção vale ouro em legibilidade: `PI * raio ** 2` comunica intenção, enquanto `3.14159 * raio ** 2` esconde um número mágico. Valores fixos espalhados pelo código viram pesadelo de manutenção; concentrados em constantes, viram documentação viva.

constantes.py

```

PI = 3.14159
TAXA_IPI = 0.25
LIMITE_ALUNOS = 60

area = PI * 10 ** 2      # 314.159

```

int — inteiros

`10 · -3 · 0 · 1_000_000`

Precisão ilimitada: cresce conforme a necessidade (2 ** 100 funciona!). O underscore é só separador visual.

float — reais

`1.68 · -0.5 · 2e3 · 3.14`

Aproximação binária, sujeita a pequenos arredondamentos. 2e3 é notação científica: 2000.0.

complex — complexos

`2 + 3j`

Parte real + imaginária, usado em cálculos científicos. O foco do curso fica em int e float.

Curiosidade que cai em prova: `0.1 + 0.2 == 0.3` devolve `False`. O resultado é `0.30000000000000004` — o float é uma representação binária e 0.1 não tem representação exata, assim como 1/3 não termina no decimal. Para valores monetários, o módulo `decimal` é a ferramenta indicada.

03 • EXPRESSÕES E OPERADORES

Expressões: código que produz um valor

Uma **expressão** é qualquer trecho de código que, ao ser avaliado, produz um valor. O literal `42` já é uma expressão — o caso mais simples. Expressões combinam **operandos** (os valores) com **operadores** (as ações): em `3 + 4 * 2`, os números são operandos e `+` e `*` são operadores. Avaliar é reduzir tudo a um único valor. Elas vivem no lado direito do `=`, dentro de `print()` e nas condições de `if`. O operador `+` também concatena textos: `"Intro" + "Python"` resulta em `"IntroPython"`.

Operador	Nome	Exemplo	Resultado
<code>+</code>	Adição	<code>7 + 2</code>	<code>9</code>
<code>-</code>	Subtração	<code>7 - 2</code>	<code>5</code>
<code>*</code>	Multiplicação	<code>7 * 2</code>	<code>14</code>
<code>/</code>	Divisão real	<code>7 / 2</code>	<code>3.5</code>
<code>//</code>	Divisão inteira	<code>7 // 2</code>	<code>3</code>
<code>%</code>	Resto da divisão	<code>7 % 2</code>	<code>1</code>
<code>**</code>	Potência	<code>7 ** 2</code>	<code>49</code>

```
python3 - interpretador
>>> 8 / 2          # sempre float!4.0>>> -7 // 2          # arredonda para BAIXO-4>>> 9 * 0.5
```

Divisão real / vs. inteira // : a barra simples sempre devolve float — até `8 / 2` resulta em `4.0`. A dupla barra descarta as casas decimais e arredonda para o **piso** (o menor inteiro possível): por isso `-7 // 2` dá `-4`, e não `-3`. Regra de bolso: `//` responde "quantas caixas cheias?" e `%` responde "quanto sobra?".

% para par ou ímpar

`numero % 2 == 0`

O teste de paridade mais usado do semestre — volta com força nos `if/else`.

% para ciclos

`indice % N`

Percorre 0..N-1 em loop: relógios, alternância, listas circulares (`13 % 12 = 1`).

% para extrair dígitos

`123 % 10 → 3`

Isola o último dígito de um número. Combine com `//` para decompor valores.

Cuidado com o `^`: em Python, `^` NÃO é potência — é XOR bit a bit (`2 ^ 3` devolve 1). Potência é `**` (`2 ** 3` devolve 8). Quem vem do Excel cai nessa. Lembre também: `2 ** 10 = 1024` — o "kilo" da computação (1 KiB).

04 • OPERADORES DE ATRIBUIÇÃO

Atribuição composta e acumuladores

Todo operador aritmético tem uma **versão composta** que compacta a atualização de uma variável pelo próprio valor dela: `x += 3` equivale exatamente a `x = x + 3`. É açúcar sintático — menos digitação e intenção mais clara. Leia sempre como "x recebe ele mesmo mais 3". A tabela abaixo mostra a família completa, sempre partindo de `x = 10`. Repare que `/=` mantém a regra da divisão real: o resultado vira float. O lado esquerdo precisa ser um nome — `3 = x` e `x + 1 = 5` são erros de sintaxe.

Operador	Exemplo	Equivale a	Resultado
<code>=</code>	<code>x = 10</code>	atribuição simples	10
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>	13
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>	7
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>	30
<code>/=</code>	<code>x /= 4</code>	<code>x = x / 4</code>	2.5
<code>//=</code>	<code>x //= 3</code>	<code>x = x // 3</code>	3
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>	1
<code>**=</code>	<code>x **= 2</code>	<code>x = x ** 2</code>	100

O caso de uso que justifica essa família é o **acumulador**: um valor que nasce neutro e é atualizado ao longo do programa. Ele será o esqueleto de médias, somas de notas, contagens e relatórios durante todo o semestre. Contadores são o mesmo padrão com passo 1 (`contador += 1`). Para **produtos**, comece com 1 (neutro da multiplicação) e acumule com `total *= fator`.

```
soma = 0      # ponto de partida: neutro da soma
soma += 4     # agora vale 4
soma += 3     # agora vale 7
soma += 10    # agora vale 17
print(soma)   # 17
```

Se esquecer a linha `soma = 0`: o primeiro `soma += 4` falha com `NameError`. A variável precisa existir antes de ser acumulada — o `+=` lê o valor atual, e não há valor algum para ler.

05 • PRECEDÊNCIA DOS OPERADORES

Precedência: a ordem que decide o resultado

Quando vários operadores aparecem juntos, o Python segue uma **ordem fixa de avaliação** — a mesma lógica da matemática escolar. Quanto mais alto o nível na tabela, mais cedo a operação é resolvida. A atribuição é sempre a última: coerente com "a direita é avaliada antes de guardar". Empate no mesmo nível? Resolva da **esquerda para a direita**. A exceção famosa é a potência, que agrupa da direita para a esquerda: `2 ** 3 ** 2` é `2 ** (3 ** 2)` = 512.

Nível	Operadores	O que faz
1°	()	Parênteses — avaliam primeiro, sempre
2°	**	Potência
3°	+x -x	Sinal unário
4°	* / // %	Multiplicativos
5°	+ -	Aditivos
6°	= += -= ...	Atribuições — por último

Resolvendo passo a passo: `2 + 3 * 4 ** 2 - 10 // 5`

O método é **reescrever**, nunca calcular tudo de uma vez. Cada reescrita preserva o valor — é isso que dá confiança.

```
2 + 3 * 4 ** 2 - 10 // 5      # 1° potência: 4 ** 2 → 16
```

```
2 + 3 * 16 - 10 // 5 # 2° multiplicativos: 48 e 22 + 48 - 2
```

```
# 3° aditivos
```

Parênteses são grátis e vencem sempre. Use-os para tornar a intenção explícita, mesmo quando não são obrigatórios: $(2 + 3) * 4 \rightarrow 20$ e $2 + 3 * 4 \rightarrow 14$ usam os mesmos números. O caso clássico é a média: `media = (n1 + n2 + n3) / 3`. Sem os parênteses, o Python calcularia `n1 + n2 + (n3 / 3)` — com `n1=6`, `n2=7` e `n3=8`, a média sairia 15.67 em vez de 7.0. Se você precisou de 3 segundos para decifrar uma expressão, ela ganha parênteses.

06 • CONSOLIDAÇÃO

Os quatro erros mais comuns de iniciante

Todos os cartões abaixo vêm de erros reais de prova. Vale ler o traceback com calma quando algo quebrar: a mensagem indica a **linha** e o **motivo** exato do problema — aprender a interpretá-la é aula disfarçada de inglês técnico. Antes de dividir por uma variável, verifique se ela pode valer zero; antes de misturar texto com número, converta com `int()` ou `str()`.

✗ = não é ==

`x = 5` guarda o valor; `x == 5` pergunta se `x` é igual a 5. Confundir os dois é o erro clássico — a comparação é tema do próximo módulo.

✗ / quando queria //

`5 / 2 → 2.5`, mas `5 // 2 → 2`. A barra simples traz decimais (float). Escolha consciente conforme o problema pedir.

✗ Divisão por zero

`10 / 0` interrompe o programa na hora com `ZeroDivisionError`. O tratamento elegante com `try/except` vem mais adiante no curso.

✗ Misturar tipos

`"2" + 3 → TypeError` (mensagem: `can only concatenate str — not int — to str`). Converta antes: `int("2") + 3 → 5`.

07 • PRÁTICA

Desafios: o que imprime?

Resolva **no papel** antes de conferir o gabarito — simular a execução na cabeça é o treino que garante nota. Depois, confira cada resposta digitando o código no interpretador (`python3` no terminal). Errar no terminal é grátis: é assim que se aprende.

DESAFIO 1**Acompanhe o valor de x linha a linha (acumulador)**

```
x = 5
x = x + 3
x *= 2
print(x)
```

GABARITO 16 — o valor evolui $5 \rightarrow 8 \rightarrow 16$; a cada linha, o valor anterior é lido e atualizado.

DESAFIO 2**Duas saídas separadas por espaço**

```
print(17 // 5, 17 % 5)
```

GABARITO 3 2 — 3 "caixas cheias" e resto 2: $17 = 5 \times 3 + 2$. O print separa os argumentos com espaço.

DESAFIO 3**Aplique a precedência antes de somar**

```
print(2 + 3 * 2 ** 2)
```

GABARITO 14 — potência primeiro ($2**2=4$), depois $3*4=12$, por fim $2+12$. Quem respondeu 20 somou antes de multiplicar.

DESAFIO 4**Divisão inteira com negativo — cuidado com o piso**

```
print(-9 // 2, -9 % 2)
```

GABARITO -5 1 — o piso de -4.5 é -5 (menor inteiro possível), e o resto completa: $-9 = 2 \times (-5) + 1$.

DESAFIO 5**Associatividade da esquerda para a direita**

```
print(10 - 4 - 3, 2 ** 3 ** 2)
```

GABARITO 3 512 — subtração resolve à esquerda: $(10-4)-3 = 3$; potência à direita: $2 ** (3 ** 2) = 2 ** 9 = 512$.

Checklist de revisão — antes da prova, você sabe...

- ✓ Ler `x = 5` como "x recebe 5" e explicar que a direita é avaliada primeiro?
- ✓ Criar nomes válidos em snake_case e citar 3 nomes proibidos (2nota, nome-aluno, class)?
- ✓ Explicar por que constantes são MAIÚSCULAS e o que são números mágicos?
- ✓ Prever o resultado dos 7 operadores com qualquer par de operandos (7 e 2 são ótimo treino)?
- ✓ Dizer de cabeça que `/` devolve float e que `//` arredonda para o piso?
- ✓ Escrever um acumulador correto, começando em 0 (ou 1, para produtos)?
- ✓ Ordenar os níveis de precedência e reescrever uma expressão passo a passo até reduzi-la?
- ✓ Diagnosticar os 4 erros clássicos: `=` vs `==`, `/` vs `//`, divisão por zero e mistura de tipos?

Plano de estudo sugerido: 15 minutos no interpretador valem mais que 1 hora de reler slides. Refaça todos os exemplos deste handout digitando (nunca copiando e colando), erre, leia o traceback e corrija. Depois, escreva 5 expressões para um colega avaliar no papel — e confirmem juntos no Python. O par autor/avaliador treina dos dois lados.